



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Operaciones Avanzadas e Integridad de Datos
Unidad de Organización Curricular:	ACADÉMICA
Nivel y Paralelo:	1 - B
Alumnos participantes:	Balarezo Pérez Diego Sebastián Bravo López Jordan Samuel Cajiao Valdivieso Paulo Alessandro Loor Velez Jhon Alejandro Pomaquero Chango Katherine Soledad
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar algoritmos de búsqueda lineal y control de límites en arreglos utilizando C++ o Java, con el fin de optimizar la localización de datos y garantizar la seguridad de la memoria.

Específicos:

- Desarrollar una búsqueda lineal en arreglos paralelos para asociar productos con sus precios, usando interrupciones eficientes (break) al hallar el elemento.
- Garantizar la integridad del programa mediante la validación estricta de los límites del ciclo for al recorrer un arreglo de datos.
- Explicar las consecuencias del desbordamiento de búfer (Buffer Overflow) mediante comentarios técnicos dentro del código fuente.

2.2 Modalidad

Detallada en la guía.

2.3 Tiempo de duración

Presenciales: Detallado en la guía

No presenciales: Detallado en la guía

2.4 Instrucciones

Para el desarrollo de las actividades propuestas por el docente es pertinente establecer cada ejercicio y los pasos a realizar para un mejor entendimiento.

El ejercicio 1 solicita desarrollar un código donde se evidencia el tema de búsqueda lineal de productos; se deberá crear un programa en C++ o Java que permita buscar productos dentro de un arreglo que contenga la información de producto utilizando el algoritmo de búsqueda lineal; también se debe tener en cuenta ciertas indicaciones al momento de la codificación: el arreglo debe tener 5 nombres de productos, crear un arreglo paralelo con los precios respectivos de cada producto, el programa deberá solicitar al usuario el nombre del producto que desea buscar, para la búsqueda del nombre se debe usar la búsqueda lineal recorriendo el arreglo, para la impresión de resultados la consola deberá mostrar el nombre del producto el precio y un mensaje de error si es que el producto no se encuentra, se debe usar el comando break para cuando se encuentre el producto.

El ejercicio 2 solicita desarrollar un código donde se evidencia el tema de prevención de “buffer overflow”; se deberá crear un programa en C++ o Java que muestre los elementos de un arreglo sin que exista un desbordamiento de memoria o mejor conocido como “buffer overflow”; también se debe tener en cuenta ciertas indicaciones al momento de la codificación: se deberá crear un arreglo de 6 espacios que contenga números enteros, se debe utilizar un bucle for para recorrer el arreglo, se debe validar correctamente los límites del arreglo sin que el programa arroje un error, al momento de la impresión se debe mostrar la posición junto con elemento correspondiente, y también se debe agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

- Analizar los ejercicios propuestos sobre búsqueda lineal y control de límites en arreglos.
- Diseñar, codificar, compilar y ejecutar los programas en C++ o Java verificando su correcto funcionamiento.
- Implementar la búsqueda de productos usando arreglos paralelos y la instrucción break.
- Validar los límites de los arreglos para evitar errores de desbordamiento (buffer overflow).
- Documentar el código explicando el funcionamiento y los posibles errores.
- Subir los archivos solicitados dentro del plazo establecido..

2.7 Resultados obtenidos

Los resultados obtenidos durante el desarrollo de los ejercicios fueron satisfactorios, ya que se logró aplicar correctamente la búsqueda lineal para localizar productos dentro de arreglos paralelos y mostrar sus respectivos precios de manera eficiente. Además, se verificó que el control adecuado de los límites en los ciclos permitió evitar errores relacionados con el acceso indebido a la memoria, garantizando así la estabilidad y seguridad del programa. También se comprendió la importancia de prevenir el desbordamiento de búfer mediante validaciones y comentarios técnicos que ayudan a mantener un código más seguro y organizado

2.8 Habilidades blandas empleadas en la práctica

- ✗ Liderazgo
- ✗ Trabajo en equipo
- ✗ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ✗ La resolución de conflictos
- ✗ Adaptabilidad
- ✗ Responsabilidad

2.9 Conclusiones

- Se logró implementar correctamente el algoritmo de búsqueda lineal en arreglos paralelos, permitiendo localizar productos y mostrar sus precios de manera rápida y organizada mediante el uso de la instrucción break.
- Se comprobó la importancia de validar los límites en los ciclos for, evitando accesos fuera del tamaño del arreglo y garantizando un manejo seguro de la memoria durante la ejecución del programa.

- Se identificó que el desbordamiento de búfer puede provocar errores, pérdida de información o fallos en el sistema, por lo que es fundamental controlar adecuadamente las posiciones de los arreglos en el código.
- El desarrollo de estos programas permitió fortalecer el uso de arreglos, ciclos y estructuras de control en C++ o Java, mejorando la lógica de programación y la seguridad en el manejo de datos.

2.10 Recomendaciones

- Usar estructuras unificadas en vez de arreglos paralelos: Reemplaza los arreglos paralelos por colecciones de objetos para evitar desincronizaciones y simplificar el manejo de los índices.
- Preferir métodos con control de rango nativo: Utiliza funciones que verifiquen los límites de memoria automáticamente al acceder a los elementos, evitando que un error de índice corrompa la memoria.
- Aplicar programación defensiva, valida siempre el tamaño y los límites de los arreglos mediante condicionales antes de recorrerlos o modificarlos, asegurando que el programa falle de forma controlada si un dato está fuera de rango.

2.11 Anexos

2.11.1 Ejercicio 1

2.11.1.1 Enunciado

Búsqueda Lineal de Productos

Desarrollar un programa en C++ o Java que permita buscar un producto dentro de un arreglo utilizando el algoritmo de búsqueda lineal.

Indicaciones:

Crear un arreglo con 5 nombres de productos.

Crear otro arreglo paralelo con sus precios.

Solicitar al usuario el nombre del producto a buscar.

Recorrer el arreglo usando búsqueda lineal.

Mostrar:

Nombre del producto.

Precio correspondiente.

Utilizar break cuando se encuentre el producto.

Mostrar un mensaje si el producto no existe.

2.11.1.2 Análisis del problema

El propósito de este programa es automatizar la localización de un producto específico y su respectivo costo a partir de una solicitud ingresada por el usuario. La estructura de almacenamiento se basa en el concepto de arreglos paralelos, lo que significa que la información está dividida en dos estructuras

unidimensionales distintas una para los nombres y otra para los precios pero vinculadas directamente a través de sus índices.

2.11.1.3 Declaración de variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
productos	string[5]	String[5]	Arreglo unidimensional con los nombres de los 5 productos.
precios	double[5] o float[5]	double[5] o float[5]	Arreglo paralelo que almacena los precios correspondientes.
productoBuscado	string	String	Almacena el nombre del producto ingresado por el usuario.
encontrado	bool	boolean	Bandera lógica inicializada en false que cambia a true si el producto existe.
i	int	int	Variable de control para iterar en el ciclo de búsqueda.

2.11.1.4 Algoritmo paso a paso

Inicio

Definir un arreglo de tipo texto con 5 nombres de productos

Definir otro arreglo de tipo real con los precios correspondientes de cada producto

Definir una variable de tipo texto para guardar el nombre del producto a buscar

Definir una variable entera para recorrer el arreglo

Definir una variable lógica para verificar si el producto fue encontrado

Asignar valores a los productos y sus precios en cada posición del arreglo

Solicitar al usuario el nombre del producto que desea buscar

Inicializar la variable encontrada en falso

Recorrer el arreglo utilizando un ciclo for desde la posición 0 hasta la posición 4

Comparar el nombre ingresado con cada producto del arreglo

Si el producto coincide mostrar el nombre del producto y su precio

Cambiar la variable encontrada a verdadero

Utilizar break para finalizar la búsqueda cuando el producto sea encontrado

Finalizar el ciclo

Verificar si la variable encontrada continúa en falso

Si no se encontró el producto mostrar un mensaje indicando que el producto no existe

Fin

2.11.1.5 Pseudocódigo

Algoritmo BusquedaLinealYControlLmites

 // Definir el tamaño del arreglo de forma estricta

 Definir TAMANIO Como Entero

 TAMANIO <- 5

```

// Declaración de los arreglos paralelos
Definir productos Como Cadena
Definir precios Como Real
Dimension productos[TAMANIO]
Dimension precios[TAMANIO]

// Inicialización de datos (Arreglos paralelos sincronizados por índice)
productos[1] <- "Laptop" ; precios[1] <- 850.00
productos[2] <- "Mouse" ; precios[2] <- 25.50
productos[3] <- "Teclado" ; precios[3] <- 45.00
productos[4] <- "Monitor" ; precios[4] <- 180.00
productos[5] <- "Audifonos"; precios[5] <- 60.00

// Variables para la búsqueda
Definir productoBuscado Como Cadena
Definir posicionEncontrada Como Entero
Definir i Como Entero

Escribir "Ingrese el nombre del producto a buscar:"
Leer productoBuscado

posicionEncontrada <- -1 // -1 significa que no ha sido encontrado

Para i <- 1 Hasta TAMANIO Con Paso 1 Hacer
    Si productos[i] = productoBuscado Entonces
        posicionEncontrada <- i
        i <- TAMANIO // Uso eficiente de interrupción al hallar el elemento (fuerza
la salida del ciclo)
    FinSi
FinPara

// Despliegue de resultados
Si posicionEncontrada <> -1 Entonces
    Escribir "¡Producto encontrado!"
    Escribir "Nombre: ", productos[posicionEncontrada]

```



```

double precios[5] = {1.50, 0.90, 0.50, 2.00, 1.20};
string buscar;
bool encontrado = false;

cout << "Ingrese el nombre del producto a buscar: ";
cin >> buscar;

for (int i = 0; i < 5; i++) {
    if (productos[i] == buscar) {
        cout << "\nProducto encontrado:\n";
        cout << "Nombre: " << productos[i] << endl;
        cout << "Precio: $" << precios[i] << endl;
        encontrado = true;
        break; // Interrumpe el ciclo al encontrar el producto
    }
}

if (!encontrado) {
    cout << "\nEl producto no existe en la lista." << endl;
}

return 0;
}

```

2.11.1.8 Prueba de escritorio

```

Ingrese el nombre del producto a buscar: Pan
Producto encontrado:
Nombre: Pan
Precio: $0.5

```

```

Ingrese el nombre del producto a buscar: Cereal
El producto no existe en la lista.

```

```

Ingrese el nombre del producto a buscar: Arroz
Producto encontrado:
Nombre: Arroz
Precio: $1.5

```

2.11.2 Ejercicio 2

2.11.2.1 Enunciado

Prevención de Buffer Overflow

Realizar un programa en C++ o Java que muestre los elementos de un arreglo sin provocar desbordamiento de memoria.

Indicaciones

Crear un arreglo con 6 números enteros.

Utilizar un ciclo for para recorrer el arreglo.

Validar correctamente los límites del ciclo.

Mostrar cada posición y su valor.

Agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

2.11.2.2 Análisis del problema

Este ejercicio se enfoca en la correcta gestión y lectura de estructuras de datos estáticas para asegurar la integridad de la memoria del sistema. Un desbordamiento de búfer en el contexto de los arreglos se produce cuando un programa intenta acceder de forma errónea a un índice que se encuentra fuera del espacio físico que le fue reservado originalmente en la memoria RAM.

2.11.2.3 Declaración de variables

Nombre de Variable	Tipo de Dato (C++)	Tipo de Dato (Java)	Descripción
números	int[6]	int[6]	Arreglo unidimensional que almacena los 6 números enteros.
Tamaño	const int	final int	Constante que define la dimensión física del arreglo (valor = 6).
i	int	int	Variable contadora del ciclo for encargada de indexar el arreglo de manera segura.

2.11.2.4 Algoritmo paso a paso

Inicio

Definir un arreglo de tipo entero con 6 posiciones

Asignar valores a cada posición del arreglo

Definir una variable entera para controlar el recorrido del ciclo

Mostrar un mensaje indicando los elementos del arreglo

Recorrer el arreglo utilizando un ciclo for desde la posición 0 hasta la posición 5

Dentro del ciclo mostrar la posición y el valor almacenado en el arreglo

Agregar un comentario indicando que si el ciclo intenta recorrer más allá de la última posición del arreglo se puede producir un desbordamiento de memoria o errores en el programa

Fin

2.11.2.5 Pseudocódigo

Algoritmo PrevencionBufferOverflow

// 1. Crear un arreglo con 6 números enteros

Definir TAMANIO Como Entero

TAMANIO <- 6

Definir numeros Como Entero

Dimension numeros[TAMANIO]

// Inicialización de los 6 elementos

numeros[1] <- 10

numeros[2] <- 20

numeros[3] <- 30

numeros[4] <- 40

numeros[5] <- 50

numeros[6] <- 60

Definir i Como Entero

Escribir "--- Recorrido Seguro del Arreglo ---"

// 2. Utilizar un ciclo for (Para) para recorrer el arreglo

// 3. Validar correctamente los límites del ciclo (de 1 hasta TAMANIO)

Para i <- 1 Hasta TAMANIO Con Paso 1 Hacer

 // 4. Mostrar cada posición y su valor

 Escribir "Posicion [", i, "]" -> Valor: ", numeros[i]

FinPara

// 5. COMENTARIO SOBRE BUFFER OVERFLOW:

// Si el ciclo intentara superar el tamaño (por ejemplo, ir hasta TAMANIO + 1),

// PSeInt lanzará un error inmediato en tiempo de ejecución ("Índice fuera de rango").

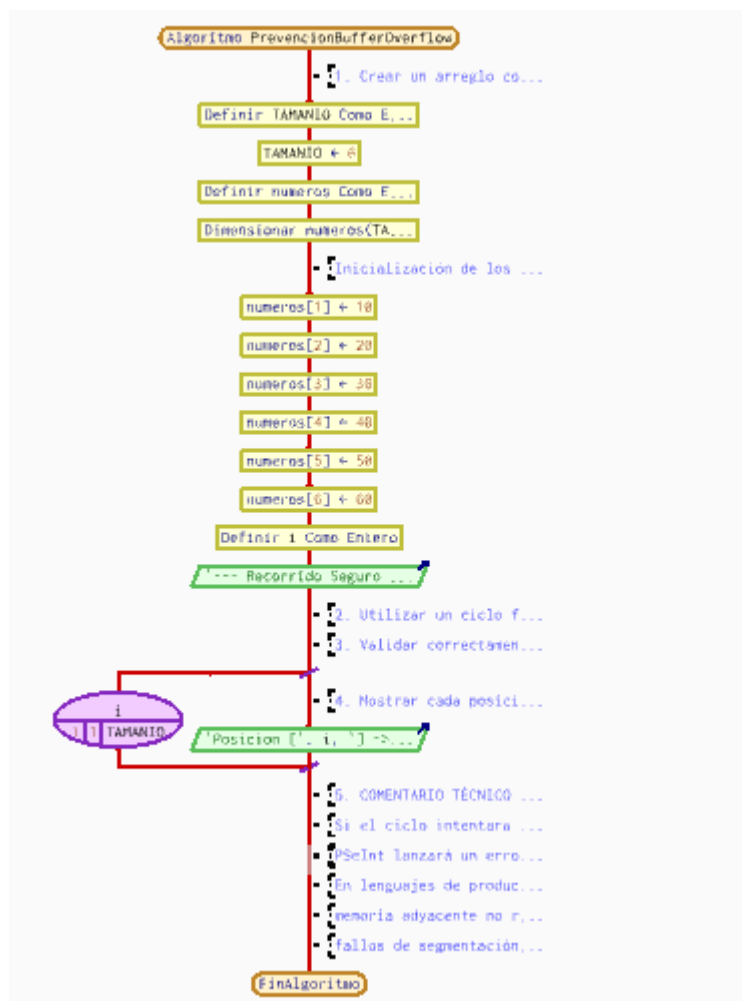
// En lenguajes de producción como C o C++, el sistema no avisa y permitiría acceder a

// memoria adyacente no reservada, lo que corrompe los datos de otras variables, causa

// fallos de segmentación (cuelgues) o genera graves brechas de seguridad informática.

FinAlgoritmo

2.11.2.6 Diagrama de flujo



2.11.2.7 Codificación C++

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int numeros[6] = {10, 20, 30, 40, 50, 60};
```

```
    cout << "Elementos del arreglo:\n";
```

```
    for (int i = 0; i < 6; i++) { // Límite correcto: i < 6
```

```
        cout << "Posicion [" << i << "] = " << numeros[i] << endl;
```

```
    }
```

```
/*
```

IMPORTANTE:

Si el ciclo for supera el tamaño del arreglo (por ejemplo $i \leq 6$ o más), se produce un desbordamiento de memoria (Buffer Overflow), lo que puede causar:

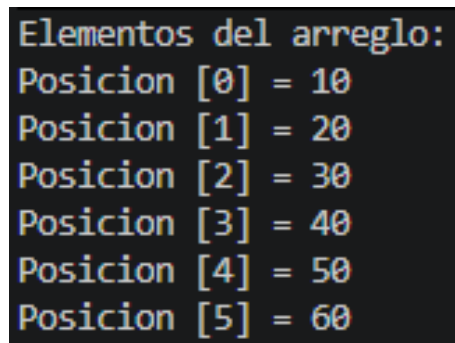
- Lectura de datos incorrectos
- Fallos en el programa
- Comportamiento inesperado

```
*/
```

```
return 0;
```

```
}
```

2.11.2.8 Prueba de escritorio

A screenshot of a terminal window showing the contents of an array. The text is as follows:

```
Elementos del arreglo:  
Posicion [0] = 10  
Posicion [1] = 20  
Posicion [2] = 30  
Posicion [3] = 40  
Posicion [4] = 50  
Posicion [5] = 60
```

2.11.2.9 Link de Github: https://github.com/TurKellou/T2.6_OperacionesAvanzadas/issues